

When is a Payment needed?

Twenty-two use cases for the durable collection object, the one question that decides whether you need it, and the cases where it should exist without ever being exposed.

Payment = durable collection obligation
Attempt = a particular execution of that payment
the single most important semantic distinction

The test

Is the intent to collect independent of any single attempt to execute it?

Internally

Always created

Externally

Progressively disclosed

Contents

1. The question that defines Payment	3
2. The use-case catalogue	3
3. Why Payment exists even when it is invisible	5
4. Authorization is not Capture	5
5. Multiple attempts	6
6. Authentication	7
7. Omnichannel	7
8. Multi-tender	8
9. Subscription	9
10. Invoice	10
11. Refunds and disputes	10
12. Agentic commerce	11
13. When Payment need not be exposed	12
14. The architecture to lock in	13

SECTION 1

The question that defines Payment

Payment is needed whenever the merchant needs a durable money-collection commitment that can survive multiple interactions, attempts or asynchronous steps.

THE TEST

Does the merchant need to maintain an ongoing intent to collect this particular amount of money, independent of any single attempt to execute it?

If yes → Payment. If no → an Attempt may be enough.

1.1 The common misunderstanding

WHAT DOES NOT REQUIRE A PAYMENT

You do **not** need a Payment object merely because you are processing a card authorization. You need one when there is value in maintaining payment state **independently from the individual execution attempt**.

1.2 The distinction in one line

Payment = what you are trying to collect
Attempt = how you tried to collect it

That separation is what allows one payment model to serve e-commerce, recurring, invoices, multi-tender, omnichannel, card present, asynchronous rails, retries, routing, marketplaces, franchises and agentic commerce — without creating a different payment model for every use case.

SECTION 2

The use-case catalogue

Twenty-two cases where a durable Payment object is required, and why.

USE CASE	WHY PAYMENT IS NEEDED	TYPICAL LIFECYCLE
Standard e-commerce checkout	Payment must survive authentication, retries and redirects	Create → Confirm → Authorize → Capture
Authorization now, capture later	Authorization and capture happen at different times	Create → Authorize → Capture

USE CASE	WHY PAYMENT IS NEEDED	TYPICAL LIFECYCLE
Partial capture	Multiple captures against one authorization	Authorize → Capture \$60 → Capture \$40
Incremental authorization	Hotel, rental or fuel needs to increase the authorized amount	\$500 auth → +\$200 increment
Delayed fulfilment	Merchant needs a payment commitment before goods or services are delivered	Payment → Fulfilment → Capture
Asynchronous payment method	Bank transfer, ACH or open banking may complete later	Created → Processing → Succeeded
3DS and customer authentication	Payment persists while the customer completes authentication	Requires action → Processing
Payment retry	The same commercial payment can have multiple attempts	Attempt 1 decline → Attempt 2 success
Smart routing	Payment can be replanned across processors and acquirers	Attempt A → retry, route B
Multi-tender checkout	Order needs multiple independent payment commitments	Order → Payment 1 + 2 + 3
Omnichannel	Payment survives movement between channels	Web → Store → Terminal
Payment link	The link or session may expire, but the underlying payment remains	Payment → Link → Customer pays
Terminal payment	A terminal attempt can fail while the payment remains open	Payment → Attempt 1 → Attempt 2
Wallet or method selection	Customer can change credential without creating a new commercial payment	Payment → PM A → PM B
Stored credential and vault	Payment references a durable credential and may need retries	Payment → Credential → Attempt
Recurring billing	Invoice needs a durable payment collection object	Invoice → Payment → Attempts
Invoice payment	Invoice can remain unpaid while collection attempts occur	Invoice → Payment → Retry
Marketplace and platform	Payment must remain independent of allocation and payout	Payment → Obligations → Payouts
Franchise	One payment may create obligations to multiple parties	Payment → Allocation → Settlement
Agentic commerce	Agent authorization, mandate and execution may be separated in time	Delegation → Payment → Attempt
Buy online, pick up in store	Payment exists before physical fulfilment	Online payment → Store fulfilment
Buy online, return in store	Original payment must remain identifiable for the refund	Original payment → CP refund

SECTION 3

Why Payment exists even when it is invisible

A simple card sale could technically be executed without a durable Payment object. We create one anyway.

Customer, merchant, card, authorization, capture. Nothing about that flow demands a durable object, and a client integrating at the simplest level will never name one.

3.1 What the Payment gives you regardless

Creating it internally provides a **stable identity** for everything that comes afterwards:

STABLE IDENTITY FOR	STABLE IDENTITY FOR
Authorization	Customer servicing
Capture	Events
Refund	Analytics
Dispute	Retries
Reconciliation	Omnichannel continuity

THE RESULTING PRINCIPLE

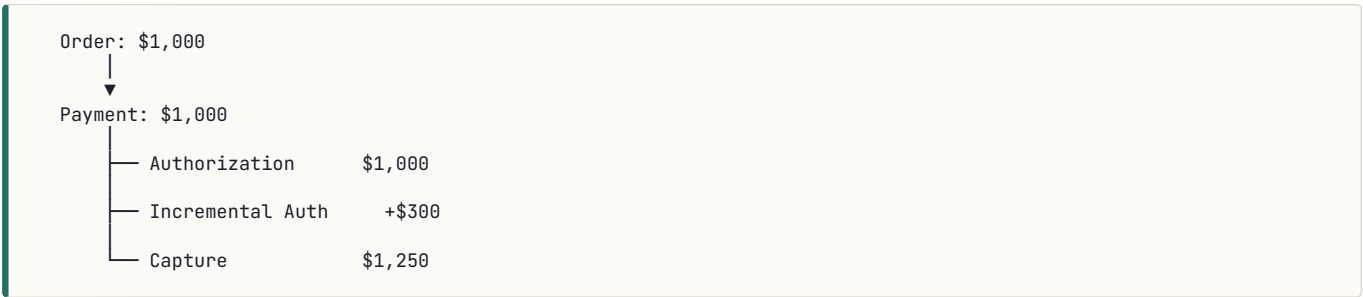
The merchant does not necessarily need to see that complexity. Payment is a first-class canonical object *internally*, and progressively disclosed *externally*. Section 13 sets out when it stays hidden.

SECTION 4

Authorization is not Capture

Probably the strongest single reason the object must exist.

4.1 Worked example — a hotel stay



The Payment gives you a durable container for the whole authorization and capture lifecycle. The authorized amount rose, the captured amount differs from both the original authorization and the total authorized, and none of that required a second payment object.

4.2 Where this pattern appears

SEGMENT	SEGMENT
Hotels	Travel
Car rental	Delayed shipment
Lodging	Marketplace fulfilment
Restaurants	
Fuel	

SECTION 5

Multiple attempts

Another fundamental reason, and the one that matters most to the orchestration strategy.

Payment PAY-123
Amount: \$500

Attempt 1

Attempt 2

Attempt 3

Visa / Acquirer A

Visa / Acquirer B

Network Token / Acquirer C

→ Declined

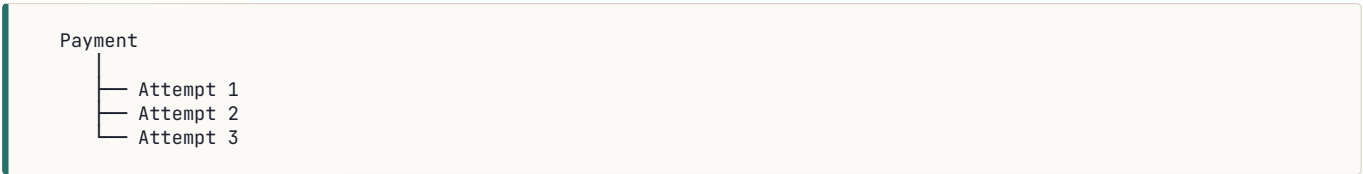
→ Declined

→ Authorized

THE REQUIREMENT

You do not want three Payments.

You want one Payment with three Attempts beneath it.

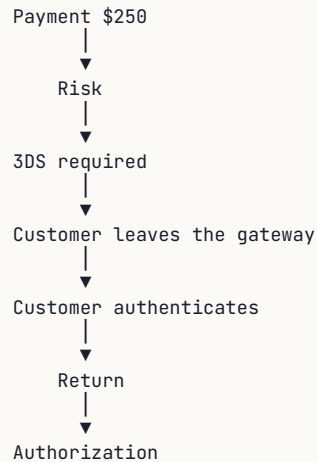


The Payment represents **what you are trying to collect**. The Attempt represents **how you tried to collect it**. That separation is what makes smart routing, replanning and retry orchestration expressible at all — without it, every route change is a new money object for the merchant to reconcile.

SECTION 6

Authentication

The payment must persist while the customer is somewhere else entirely.



THE RULE

You do not want the authentication interaction to create a new payment. The Payment persists across the entire flow, including the period where the customer is not on the merchant's site at all.

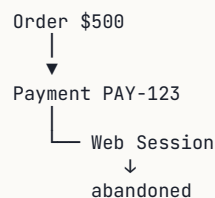
The same shape recurs for every redirect-based method, wallet handoff, open-banking consent and human-in-the-loop confirmation. One state serves all of them.

SECTION 7

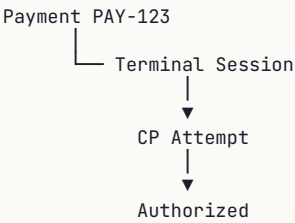
Omnichannel

One of the strongest arguments for the architecture.

The customer starts online



The customer walks into the store



WHAT THIS DEMONSTRATES

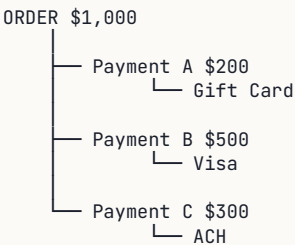
Same Payment. Different Session. Different Attempt.

This gives true channel independence — and it is only possible because the Payment is durable and the Session is not responsible for the money.

SECTION 8

Multi-tender

Each collection source has its own lifecycle, so each needs its own Payment.

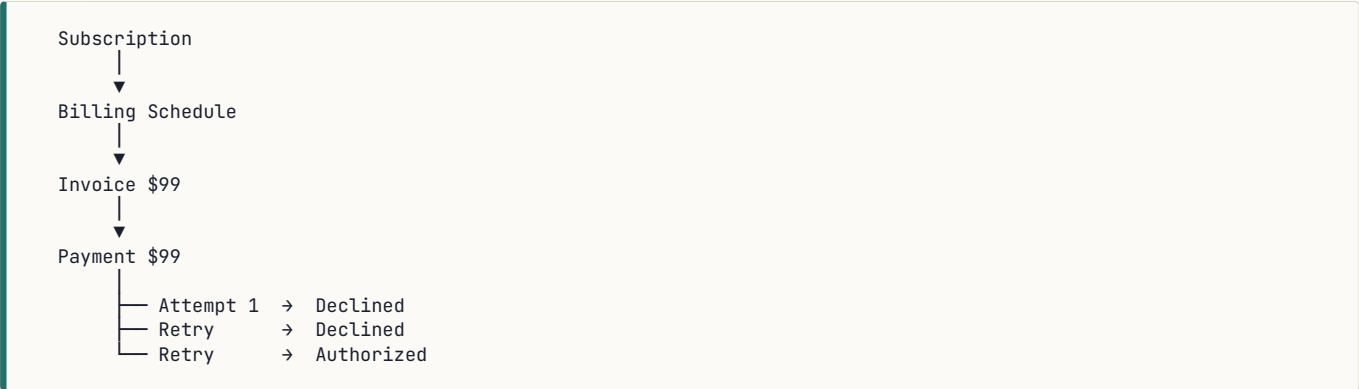


Independent Payment objects are not optional here. The gift card may settle instantly, the Visa may authorize and capture separately, and the ACH may take two days — three lifecycles that cannot share one object.

OBJECT	ANSWERS
The Order	What does the customer owe?
The Payments	How are we collecting it?

Subscription

Where Payment becomes particularly important, because three objects are competing to be "the payment" and only one of them is.

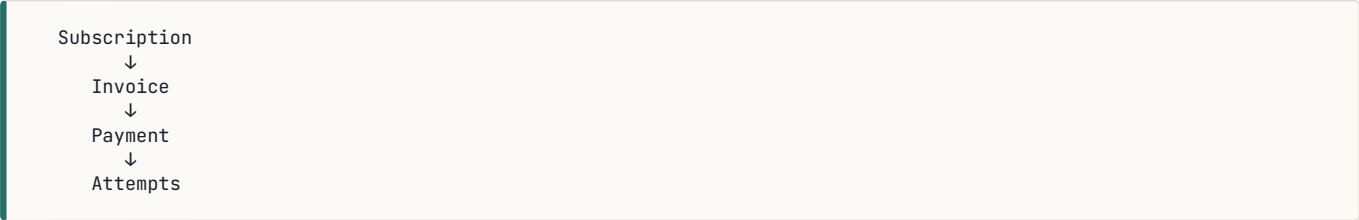


THE TWO THINGS THAT ARE NOT THE PAYMENT

The Subscription is not the Payment. It is the agreement.

The Invoice is not the Payment. It is what is owed for a period.

9.1 The chain, and who owns what



LAYER	RESPONSIBILITY
Billing	Determines <i>what</i> should be collected, and when it becomes due.
Payment	Determines the durable collection lifecycle.
Decision Engine	Determines <i>how</i> to collect it — route, credential, retry timing.

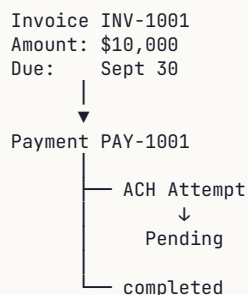
Dunning is therefore a conversation between billing and the decision engine, mediated by a Payment that outlives every individual attempt.

SECTION 10

Invoice

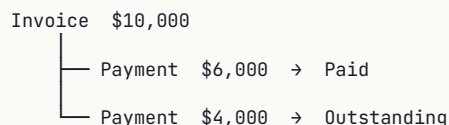
Another very strong case, and the one that forces multiple Payments against a single obligation.

10.1 Asynchronous settlement



If the ACH takes two days, you need a durable Payment. There is nothing else to hold the state during those two days, and the invoice must not be marked paid before the funds arrive.

10.2 Partial payment against an invoice



THE ARCHITECTURAL REQUIREMENT

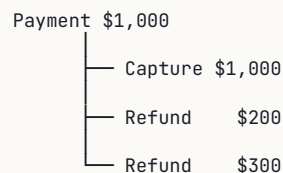
This is why the architecture must support **multiple Payments against an Invoice**, exactly as it supports multiple Payments against an Order. Invoice and Order play the same role: they state what is owed, and Payments collect it.

SECTION 11

Refunds and disputes

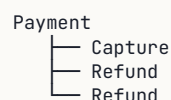
Payment gives both a stable parent, and both can occur long after the original collection.

11.1 Refund



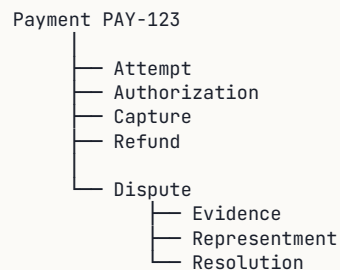
WHAT NOT TO DO

Do not set `Payment state = REFUNDED`. That loses the ability to represent multiple independent refund operations, and it collapses a money movement into a lifecycle state.



The financial truth lives in the Money and Ledger layer, not in a status field on the Payment.

11.2 Dispute

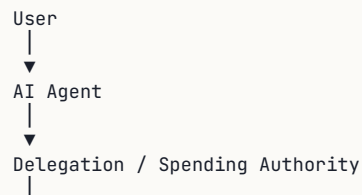


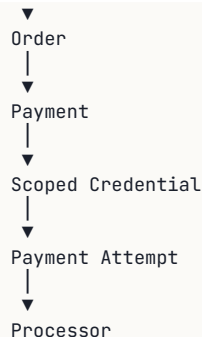
A dispute can arrive **weeks or months** after the original payment. The Payment therefore needs a durable identity that survives well past settlement — long after every Session has expired and every Attempt has closed.

SECTION 12

Agentic commerce

This becomes more important still, because authorization and execution are separated in time by design.





The agent may initiate the transaction, but execution can happen later. The Payment is the durable object linking every element of the chain of authority:

LINKED BY THE PAYMENT	LINKED BY THE PAYMENT
User authorization	Credential
Agent identity	Authentication
Merchant	Execution
Order	Money movement
Spending limit	

WHY THIS MATTERS DOWNSTREAM

When a dispute arrives on an agent-initiated transaction, this chain *is* the evidence. It exists only because the Payment held the objects together across a gap in time that no session and no attempt could span.

SECTION 13

When Payment need not be exposed

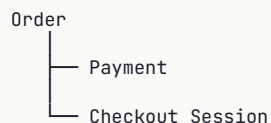
Equally important. The API must not force every developer to understand Payment.

13.1 Simple SMB checkout

```

POST /orders
POST /orders/{id}/checkout
  
```

The platform internally creates:



The merchant does not create a Payment manually, and never names one.

13.2 Simple one-click card transaction

```
POST /payments
{
  "amount": 10000,
  "currency": "USD",
  "payment_method": "pm_123",
  "confirm": true
}
```

Internally:

```
Payment
↓
Decision
↓
Attempt
↓
Authorization
↓
Capture
```

THE PRINCIPLE

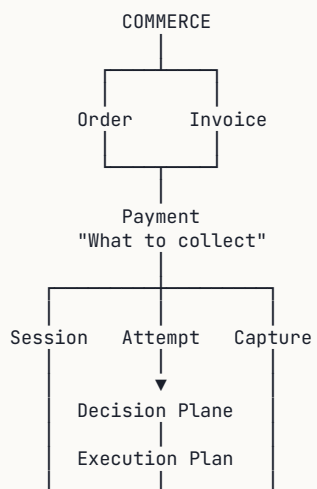
One API call. The complexity remains inside the platform.

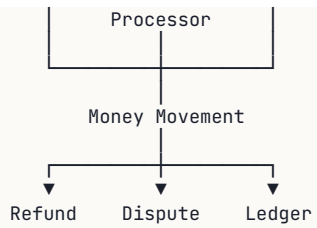
Payment is a first-class canonical object internally, and progressively disclosed externally. Simple merchants experience "Order → Checkout → Paid." Sophisticated merchants control Payment → Attempt → Decision → Capture → Refund.

SECTION 14

The architecture to lock in

The fundamental relationship this document argues for.





THE SINGLE MOST IMPORTANT SEMANTIC DISTINCTION

Payment = durable collection obligation. Attempt = a particular execution of that payment.

That distinction is what lets the platform support e-commerce, recurring, invoices, multi-tender, omnichannel, card present, asynchronous rails, retries, routing, marketplaces, franchises and agentic commerce without creating a different payment model for every use case.

14.1 The two-line summary

AUDIENCE	WHAT THEY EXPERIENCE
Simple merchants	Order → Checkout → Paid
Sophisticated merchants	Payment → Attempt → Decision → Capture → Refund

Same platform. Same objects. Different levels of disclosure.